

Packet Filtering Engine

Başak Balkan
Öğrenci No: 244410022
Kastamonu Üniversitesi
Bilgisayar Mühendisliği

Özet- Bu çalışma, Security Tools kategorisi kapsamında C++ dili ve nesneye yönelik programlama yaklaşımıyla geliştirilen bir Packet Filtering Engine uygulamasını ele almaktadır. Geliştirilen sistemde ağ paketleri kaynak IP adresi, hedef IP adresi, protokol ve port bilgilerine göre temsil edilmekte; önceden tanımlanan veya kullanıcı tarafından eklenen kurallar yardımıyla ALLOW ya da DENY kararına dönüştürülmektedir. Uygulama; Packet, Rule, FilterEngine, Statistics ve Logger sınıfları üzerine kurulmuş, böylece paket temsili, kural eşleştirme, filtreleme kararı, istatistik tutma ve loglama sorumlulukları ayrı modüller halinde tasarlanmıştır. Ek olarak web tabanlı bir arayüz hazırlanmış ve CSV dosyasından yüklenen gerçekçi paket kayıtları üzerinde toplu analiz yapılabilmektedir. Test sonuçları, sistemin hem tekli paket kontrolünde hem de veri seti tabanlı analizde beklenen filtreleme kararlarını ürettiğini göstermektedir.

Anahtar Kelimeler- packet filtering, security tools, firewall, C++, nesneye yönelik programlama, web arayüz, CSV analizi

I. GİRİŞ

Ağ güvenliği uygulamalarında paket filtreleme, gelen veya giden ağ trafiğinin belirli kurallara göre değerlendirilmesini sağlayan temel bir yaklaşımdır. Firewall sistemlerinde kullanılan bu mantık, trafiğin kaynak adresi, hedef adresi, protokol türü ve port numarası gibi alanlar üzerinden incelenmesine dayanır. NIST firewall rehberi, firewall sistemlerini farklı güvenlik düzeylerine sahip ağlar veya hostlar arasındaki ağ trafiğinin akışını kontrol eden sistemler olarak tanımlamaktadır. Bu çalışmada geliştirilen uygulama, söz konusu temel mantığı eğitim amaçlı ve OOP merkezli bir yazılım modeli olarak ele almaktadır.

Çalışmanın amacı, gerçek ağ paketlerini canlı olarak yakalayan karmaşık bir araç üretmekten ziyade, packet filtering engine kavramını anlaşılır, test edilebilir ve genişletilebilir bir C++ uygulaması olarak modellemektir. Bu nedenle paket verileri kullanıcı girişi ve CSV dosyası üzerinden alınmış; filtreleme kararları ise uygulama içinde tanımlanan kurallar ile üretilmiştir.

II. ARKA PLAN VE KULLANILAN TEKNOLOJİLER

Packet filtering yaklaşımı, IP ve taşıma katmanı protokollerinin temel başlık alanlarından yararlanır. IPv4 için kaynak ve hedef adres bilgisi RFC 791 kapsamında tanımlanan Internet Protocol yapısının temel bileşenleri arasında yer alır. TCP, güncel RFC 9293 dokümanında internet protokol yığınının önemli taşıma katmanı protokollerinden biri olarak açıklanır. UDP ise RFC 768 ile datagram temelli daha basit bir iletişim modeli olarak tanımlanmıştır. ICMP mesajları da IP üstünde hata ve kontrol mesajları taşımak için kullanılmaktadır.

Uygulamada C++ dili, sınıf tabanlı tasarım ve kapsülleme yaklaşımını desteklediği için tercih edilmiştir. Web arayüz tarafında ise HTML, CSS ve JavaScript kullanılmış; CSV dosyalarının tarayıcıda okunması için FileReader arayüzünden yararlanılmıştır. CSV biçimi, virgülle ayrılmış tablo verilerini temsil eden yaygın bir metin formatıdır ve RFC 4180 tarafından belirlenmiştir.

III. SİSTEM MİMARİSİ

Sistem iki ana katmandan oluşmaktadır. Birinci katman C++ ile geliştirilen OOP çekirdeğidir. Bu çekirdek Packet, Rule, FilterEngine, Statistics ve Logger sınıflarını içerir. İkinci katman ise kullanıcı dostu etkileşim sağlayan web arayüzüdür. C++ başlatıcı kodu, uygulama çalıştırıldığında index.html dosyasını varsayılan tarayıcıda açmakta ve arka planda OOP çekirdeğinin örnek bir filtreleme işlemiyle çalıştığını doğrulamaktadır. Windows ortamında dosya açma işlemi için ShellExecuteA işlevinden yararlanılmıştır.

Bu mimaride kurallar bellekte bir liste olarak tutulur. Her paket, sırayla kurallarla karşılaştırılır ve ilk eşleşen kuralın kararı uygulanır. Hiçbir kural eşleşmezse sistem varsayılan olarak DENY sonucunu üretir. Bu tercih güvenlik araçlarında sık kullanılan varsayılan engelleme yaklaşımına uygundur; çünkü tanımsız veya beklenmeyen trafik izinli kabul edilmez.

A. Sınıf Sorumlulukları

Packet sınıfı kaynak IP, hedef IP, protokol ve port bilgisini tutar. Rule sınıfı, bir kuralın hangi koşullarda pakete eşleşeceğini belirler. FilterEngine sınıfı kuralları saklar ve gelen paketin kararını üretir. Statistics sınıfı toplam, izin verilen ve engellenen paket sayılarını takip eder. Logger sınıfı ise filtreleme sonuçlarını log.txt dosyasına kaydeder.

IV. UYGULAMA

A. Paket ve Kural Modeli

Uygulamada her paket sourceIP, destinationIP, protocol ve port alanlarıyla temsil edilmiştir. Kurallar ise action, protocol, sourceIP, destinationIP ve port alanlarından oluşur. Kural alanlarında ANY değeri kullanıldığında ilgili alan tüm değerlerle eşleşir. Port alanında ANY karşılığı olarak -1 değeri kullanılmıştır. Bu yaklaşım, basit ancak anlaşılır bir kural eşleştirme mekanizması sağlamaktadır.

B. Filtreleme Motoru

FilterEngine sınıfı, kuralları vector yapısı içinde saklar ve evaluate fonksiyonu ile paketi sırayla tüm kurallara karşı test eder. İlk eşleşen kuralın ALLOW veya DENY değeri sonuç olarak döndürülür. Bu yapı, gerçek firewall sistemlerinde görülen kural sırası mantığının sadeleştirilmiş bir karşılığıdır.

C. Web Arayüz ve CSV Analizi

Web arayüzde tekli paket kontrolü, yeni kural ekleme, mevcut kuralları görüntüleme, istatistik gösterimi, log listesi ve CSV tabanlı toplu analiz bölümleri yer almaktadır. CSV yükleme bölümünde kullanıcı packets.csv dosyasını seçtiğinde tarayıcı dosya içeriğini metin olarak okur, her satır paket nesnesine dönüştürülür ve aynı kural eşleştirme mantığıyla analiz edilir. Bu sayede uygulama yalnızca manuel girişle değil, gerçekçi veri setleriyle de çalışabilir hale getirilmiştir.

D. Loglama ve İstatistik

Her analiz sonucunda toplam kontrol edilen paket sayısı, izin verilen paket sayısı ve engellenen paket sayısı güncellenmektedir. C++ tarafında Logger sınıfı log.txt dosyasına sonuç yazarken, web arayüz tarafında log kayıtları sayfa üzerinde listelenmektedir. Bu yapı, güvenlik araçlarında önemli olan izlenebilirlik kavramını basit bir düzeyde temsil etmektedir.

V. TEST SENARYOLARI VE SONUÇLAR

Uygulama üç temel test grubu üzerinden doğrulanmıştır. İlk olarak tekli paket kontrolünde kaynak IP adresi 192.168.1.10, hedef IP adresi 10.0.0.5, TCP protokolü ve 80 portu kullanıldığında sistem ALLOW sonucu üretmiştir. Bu sonuç, varsayılan kurallar arasında bulunan ALLOW TCP 192.168.1.10 ANY 80 kuralıyla uyumludur.

İkinci testte UDP protokolü ve 53 portu kullanılarak DNS benzeri bir paket girilmiş ve sistem DENY sonucu üretmiştir. Bu sonuç, DENY UDP ANY ANY 53 kuralının doğru çalıştığını göstermektedir. Üçüncü testte ise packets.csv dosyasındaki sekiz paket toplu olarak analiz edilmiş ve sonuçlar tablo halinde gösterilmiştir. CSV analizi sırasında istatistik değerlerinin ve log kayıtlarının güncellendiği gözlemlenmiştir.

VI. TARTIŞMA

Geliştirilen sistem, canlı ağ kartından paket yakalayan tam kapsamlı bir güvenlik ürünü değildir. Buna karşılık, packet filtering engine mantığını yazılım düzeyinde modelleyen ve OOP ilkelerini görünür hale getiren bir eğitim uygulamasıdır. Canlı trafik yakalama için Npcap benzeri ek sürücüler, yönetici yetkisi ve daha ayrıntılı protokol ayrıştırma işlemleri gerekebilir. Bu çalışmada ise stabil ve taşınabilir olması nedeniyle CSV tabanlı veri seti yaklaşımı tercih edilmiştir.

Uygulamanın güçlü yönü, filtreleme kararının ayrı sınıflar ve açık sorumluluklarla modellenmesidir. Kapsülleme sayesinde paket ve kural verileri sınıflar içinde tutulmuş, karar üretimi FilterEngine sınıfına ayrılmıştır. Böylece sistem ileride IP aralığı desteği, port aralığı, kural önceliği, zaman damgası veya protokol genişletmeleri gibi özelliklerle geliştirilebilir.

VII. SINIRLILIKLAR VE GELECEK ÇALIŞMALAR

Mevcut uygulama IP adreslerini metinsel eşitlik üzerinden karşılaştırmaktadır; CIDR aralıkları veya alt ağ maskesi hesaplaması yapmamaktadır. Ayrıca paket içeriği, TCP bayrakları veya bağlantı durumu gibi gelişmiş alanlar değerlendirilmemektedir. Gelecek çalışmalarda IP aralığı desteği, port aralığı tanımlama, kural önceliği, CSV dışı aktarma ve daha ayrıntılı raporlama özellikleri eklenebilir. Daha ileri bir aşamada gerçek ağ trafiğini yakalayabilen bir kütüphane ile entegrasyon kurulabilir.

VIII. SONUÇ

Bu çalışmada C++ ve nesneye yönelik programlama yaklaşımı kullanılarak Security Tools kategorisine uygun bir Packet Filtering Engine uygulaması geliştirilmiştir. Uygulama, ağ paketlerini kaynak IP, hedef IP, protokol ve port bilgilerine göre değerlendirmekte; tanımlı kurallara göre ALLOW veya DENY kararı üretmektedir. Web arayüz ve CSV tabanlı toplu analiz desteği, sistemin kullanımını daha anlaşılır ve gerçekçi hale getirmiştir. Elde edilen sonuçlar, geliştirilen uygulamanın packet filtering mantığını temel düzeyde doğru şekilde modellediğini ve OOP ilkelerini proje içinde uygulanabilir biçimde gösterdiğini ortaya koymaktadır.

KAYNAKÇA

- [1] K. Scarfone and P. Hoffman, "Guidelines on Firewalls and Firewall Policy," NIST Special Publication 800-41 Revision 1, National Institute of Standards and Technology, 2009. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/41/r1/final>
- [2] J. Postel, "Internet Protocol," RFC 791, Internet Engineering Task Force, 1981. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc791>
- [3] M. Eddy, Ed., "Transmission Control Protocol (TCP)," RFC 9293, Internet Engineering Task Force, 2022. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc9293>
- [4] J. Postel, "User Datagram Protocol," RFC 768, Internet Engineering Task Force, 1980. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc768>
- [5] J. Postel, "Internet Control Message Protocol," RFC 792, Internet Engineering Task Force, 1981. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc792>
- [6] Y. Shafranovich, "Common Format and MIME Type for Comma-Separated Values (CSV) Files," RFC 4180, Internet Engineering Task Force, 2005. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc4180>
- [7] Microsoft, "ShellExecuteA function (shellapi.h)," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/api/shellapi/nf-shellapi-shellexecutea>
- [8] MDN Web Docs, "FileReader: readAsText() method," Mozilla Developer Network. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/FileReader/readAsText>
- [9] B. Stroustrup, The C++ Programming Language, 4th ed. Boston, MA, USA: Addison-Wesley, 2013.
- [10] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Boston, MA, USA: Addison-Wesley, 1994.
- [11] W. R. Stevens, TCP/IP Illustrated, Volume 1: The Protocols. Boston, MA, USA: Addison-Wesley, 1994.
- [12] M. Sikorski and A. Honig, Practical Malware Analysis. San Francisco, CA, USA: No Starch Press, 2012.